

Universidad de Granada

Grado en Ingeniería Informática

Práctica 5: Algoritmos de Programación Dinámica

Algorítmica | Grupo C1-3

Problema 1

Autores:

- Omar Segura Arévalo
- Marvin Augusto Fonseca Zamora
- Jaime Espín Rodríguez
- Francisco González Bailón
-

Alejandro Moreno Millán

Curso Académico: 2025-2026

Memoria: Resolución del Problema de la Compra Óptima con Programación Dinámica

1. Descripción del Problema Asignado

Enunciado

Disponemos de K euros para hacer la compra y tenemos una lista de n posibles productos que podemos comprar. Cada producto i tiene un precio, $p(i)$ (que será siempre un número entero), y una utilidad, $u(i)$. De cada producto estamos dispuestos a comprar como máximo 2 unidades. Pero hay una oferta 3x2 en todos los productos, de manera que si compramos 2 unidades de un producto nos regalan una tercera unidad. Queremos elegir los productos a comprar, y cuántos de cada tipo, maximizando la utilidad de los productos comprados.

Ejemplo: Para un problema con $n = 3$, $K = 10$, $p = (3,4,5)$ y $u = (7,8,9)$, la solución óptima consiste en llevarnos 3 unidades del primer producto y una unidad del segundo, con un precio total de 10 y una utilidad de 29.

Tarea: Diseñar e implementar un algoritmo de Programación Dinámica que resuelva este problema, indicando la utilidad máxima obtenida, cuáles tipos de productos adquirimos y cuántos de cada tipo. Determinar cuál es su eficiencia teórica. Aplicarlo para resolver el ejemplo anterior y un caso adicional del problema, construyendo la(s) tabla(s) correspondiente(s).

Resumen del Problema

Contamos con un presupuesto máximo y una lista de productos con un precio y una utilidad determinada. El objetivo es maximizar la utilidad total comprando como máximo 2 unidades pagadas de cada producto, aprovechando que existe una oferta 3x2 (si pagamos 2, nos llevamos 3 unidades).

2. Planteamiento mediante Programación Dinámica

2.1 Planteamiento de la solución como secuencia de decisiones

El problema se divide en n etapas, una por cada producto. En cada etapa, debemos decidir cuántas unidades del producto actual vamos a comprar teniendo en cuenta el dinero que nos queda. Las opciones son:

- No comprar el producto (0 unidades)
- Comprar 1 unidad pagando su precio
- Comprar 2 unidades pagando el doble de su precio, lo que nos da 3 unidades por la oferta

2.2 Verificación del principio de optimalidad

Para que nuestra compra final sea la más óptima, cualquier decisión intermedia que tomemos con los primeros i productos y un presupuesto k debe ser también la mejor combinación posible para ese subproblema. Si encontramos la mejor forma de gastar el presupuesto en los primeros $i-1$ productos, al añadir el producto i evaluamos si mejora o no la utilidad máxima ya calculada.

2.3 Definición recursiva del valor de la solución óptima

Definimos $T(i, k)$ como la utilidad máxima que podemos obtener considerando los primeros i productos con un presupuesto disponible k . La ecuación de recurrencia es la siguiente:

$$T(i, k) = \max \begin{cases} T(i-1, k) & \text{(No comprar)} \\ T(i-1, k - p_i) + u_i & \text{si } k \geq p_i \text{ (Comprar 1)} \\ T(i-1, k - 2p_i) + 3u_i & \text{si } k \geq 2p_i \text{ (Comprar 2 y llevar 3)} \end{cases}$$

Explicación de los casos:

- **T(i-1, k):** Representa la opción de no comprar el producto i . La utilidad acumulada no varía y el presupuesto disponible k se mantiene intacto para los primeros $i-1$ productos.
- **T(i-1, k - p_i) + u_i:** Modela la decisión de comprar exactamente 1 unidad. Se añade la utilidad de dicha unidad (u_i) y se reduce el presupuesto disponible en el coste de un solo producto (p_i).
- **T(i-1, k - 2p_i) + 3u_i:** Justifica matemáticamente la oferta 3x2 del enunciado. Si decidimos adquirir el producto aprovechando la promoción, pagamos un máximo de 2 unidades (reduciendo el presupuesto en $2p_i$). A cambio, el problema nos regala una tercera unidad, lo que significa que nos llevamos 3 unidades completas y sumamos el triple de su utilidad ($3u_i$) pagando solo el precio de dos. El algoritmo evalúa mediante la función max cuál de estas tres decisiones alternativas proporciona el mayor beneficio.

Donde p_i es el precio del producto i y u_i es su utilidad o beneficio. El caso base es **T(0, k) = 0** y **T(i, 0) = 0**, que en el código se maneja inicializando la matriz entera a 0.

2.4 Cálculo del valor de la solución óptima (enfoque ascendente)

Creamos una matriz (tabla) de dimensiones $(n+1) \times (K+1)$. Con dos bucles anidados, iteramos primero por cada producto (filas) y luego por cada posible cantidad de dinero desde 1 hasta el presupuesto máximo (columnas). Para cada celda, calculamos el máximo de las tres decisiones posibles apoyándonos en los resultados de la fila anterior (que ya están calculados), evitando así recálculos redundantes.

2.5 Determinación de la solución óptima (uso de tablas)

Una vez rellena la tabla, la utilidad máxima estará en la celda `tabla[n][K]`. Para saber qué productos hemos comprado, usamos la función auxiliar `construirSolucion`. Empezamos en la última celda y vamos hacia atrás:

- Si el valor de la celda actual es igual al de la celda justo encima (`tabla[i-1][k]`), significa que no compramos el producto i . Subimos una fila.
- Si el valor coincide con el de la oferta (`tabla[i-1][k - 2p] + 3u`), añadimos 3 unidades a la solución, restamos el coste de 2 unidades al presupuesto y subimos.
- En caso contrario, añadimos 1 unidad, restamos su coste al presupuesto y subimos.

3. Diseño del Algoritmo y Eficiencia

3.1 Especificación del algoritmo

Para construir el algoritmo principal hemos desarrollado una función denominada `mejorCombinacionCompra`. La función recibe por parámetros el dinero máximo disponible, el número de productos, vector de precios y un vector con el beneficio que nos otorga cada producto.

El primer paso del algoritmo es definir la tabla de programación dinámica, inicializándola a 0:

```
vector<vector<int>>> tabla(n_productos + 1, vector<int>(dinero_disponible + 1, 0));
```

Posteriormente hay que recorrer de uno en uno el número de productos que tenemos disponibles, así como recorrer también de uno en uno nuestro dinero disponible. Esto nos lleva a realizar un doble bucle `for`. Obtenemos el precio y el beneficio de la iteración actual, y la primera pregunta esencial es ¿tenemos dinero para comprar aunque sea una unidad de este producto? Si la respuesta es afirmativa, tenemos tres caminos distintos, y si la respuesta es negativa es que no podemos directamente comprar el producto.

```
for (int i = 1; i <= n_productos; i++) {
    for (int k = 1; k <= dinero_disponible; k++) {

        int p_actual = precio[i-1];
        int beneficio_actual = beneficio[i-1];

        if (p_actual <= k) { // ¿Hay dinero para aunque sea uno?
            ...
        } else { // No hay dinero
```

```

        tabla[i][k] = tabla[i-1][k];
    }
}

```

Si tenemos dinero para comprar aunque sea una unidad del producto tenemos que decidir qué es mejor opción:

1. **No comprarlo (0 unidades):** Si no compramos este producto, es como si no tuviéramos dinero, por lo que el valor de la tabla tiene que ser de $i-1$ porque no estamos comprando el producto actual y con el mismo dinero máximo porque no hemos gastado.
2. **Comprar una unidad:** Si compramos el producto, tenemos que tener en cuenta que como lo estamos comprando estamos obteniendo un beneficio `beneficio_actual` y además hay que tener en cuenta lo que podremos obtener en el futuro con los productos restantes $i-1$ y con el dinero disponible $k-p_{\text{actual}}$.
3. **Comprar dos unidades (oferta):** Para poder optar a este camino, primero el algoritmo debe comprobar mediante una condición si tenemos dinero suficiente para pagar dos unidades del producto actual ($2 * p_{\text{actual}} \leq k$). Si disponemos de ese dinero, calculamos el beneficio sumando la utilidad de tres unidades (ya que pagamos dos y nos regalan la tercera, es decir, $3 * \text{beneficio_actual}$) más el beneficio óptimo que ya teníamos calculado para los productos anteriores ($i-1$) utilizando el presupuesto que nos sobraría tras pagar esas dos unidades ($k - 2 * p_{\text{actual}}$). Si no tenemos presupuesto suficiente para pagar las dos unidades, esta opción se descarta asignándole un valor de 0.

Finalmente, actualizamos el valor de la celda actual de la tabla (`tabla[i][k]`) quedándonos con el valor máximo entre las tres opciones evaluadas mediante la función `max()`.

```

int no_comprar = tabla[i-1][k];
int comprar_uno = tabla[i-1][k-p_actual] + beneficio_actual;
int comprar_dos = (2 * p_actual <= k) ? tabla[i-1][k - 2 * p_actual] + 3 * beneficio_actual : 0;
tabla[i][k] = max({no_comprar, comprar_uno, comprar_dos});

```

Además, el algoritmo realiza dos operaciones más mediante dos funciones auxiliares. Lo primero es mostrar la tabla, lo que implica simplemente recorrer los vectores e imprimirlos. Y la otra operación más interesante es `construirSolucion`.

En lugar de calcular nuevos valores, esta función rastrea la tabla hacia atrás (desde la última celda calculada hasta el principio) para ver qué decisiones tomamos. Empezamos con un bucle `for` que recorre los productos desde el último hasta el primero, y que se detiene si nos quedamos sin presupuesto:

```

for (int i = n_productos; i > 0 && dinero_disponible > 0; --i) {
    const int p = precio[i-1];
    const int b = beneficio[i-1];
}

```

Dentro del bucle, para saber qué hicimos con cada producto, comparamos el valor de nuestra celda actual con la celda de la fila superior ($i-1$). Tenemos tres casos (como antes):

Opción 1 (No se compró): Si el valor de la celda es exactamente igual al de la celda de arriba, significa que este producto no aportó nada nuevo. Hacemos un `continue` para saltarlo y el presupuesto se queda igual.

```

if (tabla[i][dinero_disponible] == tabla[i-1][dinero_disponible]) continue;

```

Opción 2 (Se aprovechó la oferta): Si el valor de la celda coincide con la fórmula de la oferta (es decir, el valor óptimo anterior restando el precio de 2 y sumando el beneficio de 3), significa que elegimos este camino. Metemos 3 unidades en el vector de compras y le restamos el precio de 2 unidades a nuestro presupuesto disponible.

```

else if (2 * p <= dinero_disponible && tabla[i][dinero_disponible] == tabla[i-1][dinero_disponible] -
        compras.emplace_back(i, 3);

```

```

    dinero_disponible -= 2 * p;
}

```

Opción 3 (Se compró 1 unidad): Si no se cumple ninguna de las dos condiciones anteriores, por descarte sabemos que compramos una sola unidad. Añadimos 1 unidad a las compras y restamos su precio al dinero disponible.

```

else {
    compras.emplace_back(i, 1);
    dinero_disponible -= p;
}

```

Al terminar de recorrer la tabla, devolvemos el vector `compras` que ya contiene la lista definitiva de los productos elegidos y sus cantidades.

3.2 Eficiencia teórica

- **Complejidad temporal:** $O(n \times K)$ donde n es el número total de productos y K es el dinero máximo disponible. Esto se debe a los dos bucles `for` anidados que recorren la matriz. La función de reconstrucción cuesta $O(n)$, por lo que no altera la eficiencia asintótica.
- **Complejidad espacial:** $O(n \times K)$ porque necesitamos almacenar una matriz de tamaño $(n+1) \times (K+1)$ de enteros en memoria para guardar las soluciones a los subproblemas.

4. Ejemplos de Aplicación y Casos de Prueba

4.1 Resolución del ejemplo base

Datos del problema: $n = 3$, $K = 10$, $p = (3, 4, 5)$, $u = (7, 8, 9)$

Tabla de Programación Dinámica $T(i,k)$

i \ k	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	7	7	7	21	21	21	21	21
2	0	0	0	7	8	8	21	21	24	24	29
3	0	0	0	7	8	9	21	21	24	24	29

Compra Óptima 1

Producto 2: 1 unidad(es)

Producto 1: 3 unidad(es)

Utilidad máxima obtenida: 29

Análisis de la solución Observando la tabla, partimos de la celda final $T(3, 10) = 29$. Como este valor es igual al de la celda superior $T(2, 10) = 29$, deducimos que no se compra el Producto 3. En la fila 2, el valor 29 se obtiene de comprar 1 unidad del Producto 2 (coste 4, utilidad 8), por lo que restamos 4 al presupuesto y pasamos a $T(1, 6) = 21$. Este valor proviene de la oferta 3x2 del Producto 1 (pagamos 2 unidades con coste 6, ganamos 21 de utilidad). El presupuesto queda a 0.

4.2 Resolución de caso adicional

Datos del problema: $n = 6$, $K = 16$, $p = (1, 2, 3, 4, 5, 6)$, $u = (7, 8, 9, 5, 6, 18)$

Tabla de Programación Dinámica $T(i,k)$

i \ k	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	7	21	21	21	21	21	21	21	21	21	21	21	21	21	21	21
2	0	7	21	21	29	31	45	45	45	45	45	45	45	45	45	45	45
3	0	7	21	21	29	31	45	45	48	54	56	58	72	72	72	72	72
4	0	7	21	21	29	31	45	45	48	54	56	58	72	72	72	72	77
5	0	7	21	21	29	31	45	45	48	54	56	58	72	72	72	72	77
6	0	7	21	21	29	31	45	45	48	54	56	58	72	72	75	75	83

Compra Óptima 2

Producto 6: 3 unidad(es)

Producto 2: 1 unidad(es)

Producto 1: 3 unidad(es)

Utilidad máxima obtenida: 83

Análisis de la solución Comenzamos en la celda $T(6, 16) = 83$. Este valor se forma al pagar 2 unidades del Producto 6 (coste 12) para llevarse 3 gracias a la oferta, lo que suma 54 de utilidad. Nos quedan 4 euros. En $K=4$, el valor máximo es 29, que se arrastra desde la fila 2. Por tanto, no compramos los productos 5, 4 ni 3. En $T(2, 4) = 29$, compramos 1 unidad del Producto 2 (coste 2, utilidad 8). Con los 2 euros restantes, subimos a la fila 1 y pagamos 2 unidades del Producto 1 (coste 2) para llevar 3, sumando los últimos 21 de utilidad.

5. Conclusión

Esta práctica nos ha servido para consolidar nuestro conocimiento sobre el diseño y la implementación de algoritmos basados en Programación Dinámica. A lo largo del desarrollo, hemos podido comprobar cómo la descomposición de un problema complejo en subproblemas más pequeños nos permite asegurar una solución óptima de manera estructurada.

Queremos resaltar que hemos interiorizado el valor de generar y rellenar una matriz de estados (la tabla dinámica). Hemos visto cómo, en lugar de recalcular valores repetidamente o usar técnicas exhaustivas que dispararían el tiempo de ejecución, podemos explotar la memoria espacial para optimizar el tiempo. El hecho de reutilizar la propia tabla completada para recorrerla hacia atrás y reconstruir las decisiones exactas que tomamos demuestra cómo se puede obtener el recorrido óptimo de una forma mucho más eficiente y elegante.